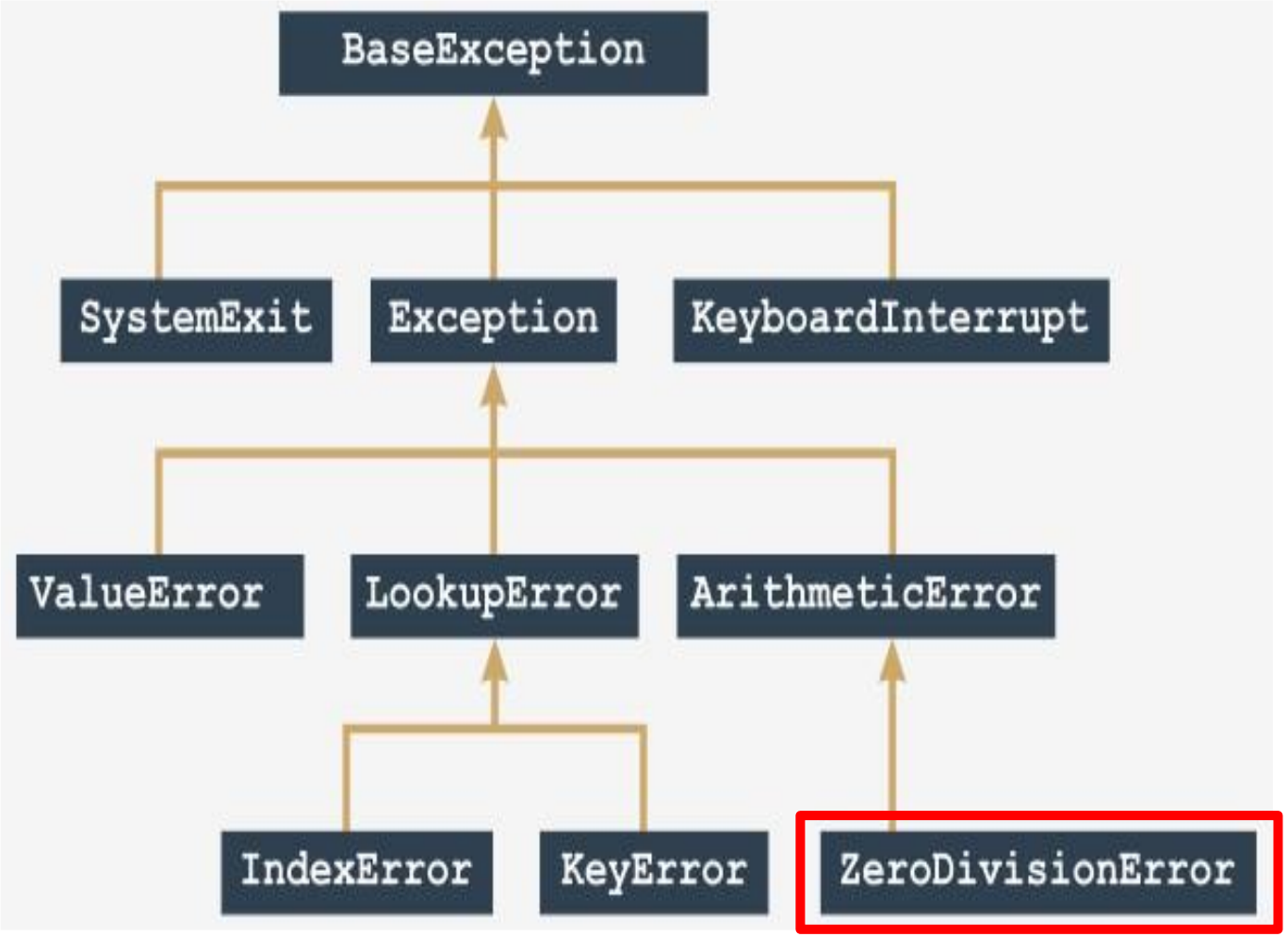
5. Gyakorlat

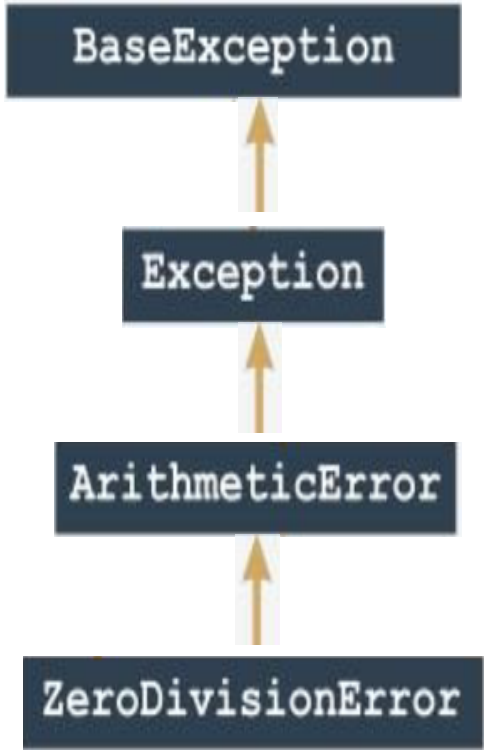
Python kurzus

Kivételkezelés
OOP bevezetés

Kivételek - Exceptions

A Python3 beépített kivételei **osztályok**, amelyek **fa alakú hierarchiát** alkotnak:





Példák a kivételekről

1. Exception osztályok:

```
try:
    i = int("Hello!")
    #print(1/0)
except Exception as e:
    print(e)
    print(e.__str__())
    print(e.__class__.__name__)
```

```
invalid literal for int() with base 10: 'Hello!'
invalid literal for int() with base 10: 'Hello!'
ValueError
```

```
division by zero
division by zero
ZeroDivisionError
```

- Az **Exception** az összes beépített kivétel alap osztálya
- Az **e** változó tartalmazza a kivétel objektumát
- **print(e)** kiírja a kivétel alapértelmezett üzenetét
- **print(e.__str__())** az **e** kivétel __str__ metódusa szintén a kivétel üzenetét írja ki

2. Az Exception class tree kiírása:

```
def print_exception_tree(thisclass, nest = 0):
    if nest > 1:
        print(" |"*(nest-1),end="")
    if nest > 0:
        print(" +------",end="")
        print(thisclass.__name__)
    for subclass in thisclass.__subclasses__():
        print_exception_tree(subclass, nest + 1)

print_exception_tree(BaseException)
#print_exception_tree(Exception,1)      # uaz
#print_exception_tree(ArithmeticError,1)
```

```
+------Exception
| +------TypeError
| +------StopAsyncIteration
| +------StopIteration
| +------ImportError
| | +------ModuleNotFoundError
| | +------ZipImportError
| +------OSError
| | +------ConnectionError
…
```

```
+------ArithmeticError
| +------FloatingPointError
| +------OverflowError
| +------ZeroDivisionError
```

Kivételek kiváltása: a raise utasítás

A **raise** utasítás az adott **exception** kivételt úgy fogja kiváltani, mintha az a szokásos (természetes) módon keletkezett volna:

Hiba esetén:

```
def próba(n):
    try:
        return 1 / n
    except ArithmeticError:
        print("Aritmetikai probléma!")
    return None
próba(0)                #a hiba kiváltása
print("vége")
```

```
Aritmetikai probléma!
vége
```

A **raise** alkalmazásával:

```
def próba(n):
    raise ZeroDivisionError

try:
    próba(2)     #nem okozna hibát
except ArithmeticError:
    print("Mi történt? Egy hiba?")
print("vége")
```

```
Mi történt? Egy hiba?
vége
```

Példa a **raise** utasításra

```
def próba(n):
    try:
        return n / 0
    except:
        print("Megint egy hiba!")
        raise

try:
    próba(0)
except ArithmeticError:
    print("Igen, ez egy aritmetikai hiba.")

print("vége")
```

```
Megint egy hiba!
Igen, ez egy aritmetikai hiba.
vége
```

A **raise** nélkül:

```
Megint egy hiba!
vége
```

Az assert kifejezés

```
import math
x = float(input("Kérek egy számot: "))
assert x >= 0.0
x = math.sqrt(x)
print(x)
```

```
Kérek egy számot: -1
Traceback (most recent call last):
  File "4ea_proba.py", line 4, in <module>
    assert x >= 0.0
AssertionError
```

```
import math
while True:
    x = float(input("Kérek egy számot: "))
    try:
        assert x >= 0.0
        break
    except AssertionError:
        print("A szám negatív!")
x = math.sqrt(x)
print(x)
```

```
Kérek egy számot: 4
2.0
```

```
Kérek egy számot: -1
A szám negatív!
Kérek egy számot: -2
A szám negatív!
Kérek egy számot: 0
0.0
```

Példák beépített kivételekre

- **ArithmeticError**
  Aritmetikai műveletek által okozott kivételek, mint a nullával osztás vagy egy argumentum érvénytelen tartománya.
- **AssertionError**
  Az assert utasítás sikertelen.
- **AttributeError**
  Helytelen attribútumra való hivatkozás vagy értékadás.
- **BaseException**
  A legáltalánosabb kivétel, az **except** és **except BaseException** kifejezések egyenértékűek.
- **IndexError**
  Nem található egy index a sorozatban.
- **KeyError**
  Nem létező dictionary kulcsra hivatkozunk.

- **NameError**

  Nem meghatározott lokális vagy globális név keresésekor.
- **SyntaxError**

  Az elemző szintaktikai hibát talált.
- **TypeError**

  Beépített operátornak vagy függvénynek helytelen típust adunk át.
- **ValueError**

  Argumentum hiba esetén, melyet a TypeError nem tartalmaz, ha argumentumokként érvénytelen adatokat adtunk meg.
- **ZeroDivisionError**

  Osztásnál vagy maradék (modulo) műveletnél 0 második argumentum esetében.
- **KeyboardInterrupt**

  A felhasználó megszakítja a program végrehajtását

Hibakezelési példák

Hibák az adatokban és hibák a kódban – a tesztelés kötelező

- **Érték hiba, attribútum hiba, szintaktikai hiba példák:**

```
x = int(input('Kérek egy egész számot: '))
```

12o

```
lst=[]
lst.appendd(1)
```

```
def sz(a=2,b):
    return a*b
print(sz(3,4))
```

- **Kódolási hibák:**

```
for i in range(5):
    if i % 2 != 0:
        print(i, 'páros')
    else:
        print(i,'páratlan')
```

```
Sugár = 10
terület =  sugár**2*3.14
```

```
lista = []
a = lista[-1]
```

```
x = 2.
y = '3'
print(x + y)
```

Megoldások:

```
x = int(input('Kérek egy egész számot: '))
```

Kérek egy egész számot: 12o
ValueError: invalid literal for int(): '12o'

```
lst=[]
lst.appendd(1)
```

AttributeError: 'list' object has no attribute 'appendd'.

```
def sz(a=2,b):
    return a*b
print(sz(3,4))
```

SyntaxError: non-default argument follows default argument

```
Sugár = 10
terület =  sugár**2*3.14
```

NameError: name 'sugár' is not defined.

```
for i in range(5):
    if i % 2 != 0: print(i, 'páros')
    else: print(i,'páratlan')
```

0 páratlan
1 páros
2 páratlan
3 páros
4 páratlan

```
x = 2.
y = '3'
print(x + y)
```

TypeError: unsupported operand type(s) for +: 'float' and 'str'

```
lista = []
a = lista[-1]
```

IndexError: list index out of range

```
try:
    print(1/0)
except ZeroDivisionError:
    print("zero")
except ArithmeticError:
    print("arith")
except:
    print("end")
```

```
try:
    print(1/0)
except ArithmeticError:
    print("arith")
except ZeroDivisionError:
    print("zero")
except:
    print("end")
```

```
def rec(x):
    assert x
    return 1/x

try:
    print(rec(0))
except ZeroDivisionError:
    print("zero")
except:
    print("end")
```

Mi a felmerülő kivétel neve?

Mi a legáltalánosabb Python kivétel neve?

A class szó lehet-e egy tetszőleges osztály neve?

1. feladat: Beolvasás hibakezelése

```
def read_int(szöveg, min, max):
   ###
v = read_int("Kérek egy egész számot -10 és 10 között:", -10, 10)
print("A szám:", v)
```

```
Kérek egy egész számot -10 és 10 között: 100
Hiba: a szám nem a  -10…10 tartományban van
Kérek egy egész számot -10 és 10 között: qwe
Hiba: csak számjegy adható meg
Kérek egy egész számot -10 és 10 között: 4.5
Hiba: csak számjegy adható meg
Kérek egy egész számot -10 és 10 között: 2
A szám: 2
```

HF. Ellenőrizzük a float beolvasást is!

4.2 → Hiba: csak egész szám adható meg

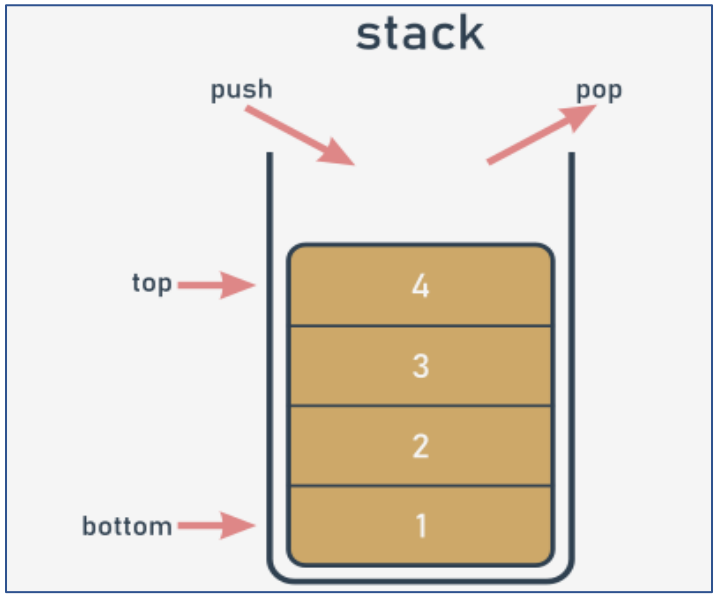
4.0 → A szám: 4

OOP – Object Oriented Programming I.

```
class FirstClass:
pass
```

```
elso_object = FirstClass()
```

Példa: Verem - procedurális módszerrel




```
stack = []

def push(val):
   stack.append(val)

def pop():
   val = stack[-1]
   del stack[-1]
   return val

push(3)
push(2)
push(1)

print(pop())
print(pop())
print(pop())
```

Verem – objektum orientált módon

```
class Stack:                                  # a Stack osztály definíciója
   def __init__(self):          # a konstruktor függvény definíciója
      self.stack_list = []                    # attribútum hozzáadása

stack_object = Stack()                        # az objektum létrehozása
print(len(stack_object.stack_list))           # az attribútum elérése
```

- Ha két _ _ kezdi a változónevet (double underscores = dunder), akkor ez **privát változó** lesz – vagyis az osztályon belülről érhető csak el.
- Ezt **encapsulation**-nak nevezzük.
- **AttributeError** lép fel, ha kívülről hivatkozunk rá.
-
```
def leng(self):    # a verem hosszának lekérdezése
    val = len(self.__stack_list)
    return(val)
```

```
class Stack:
  def __init__(self):
     self.__stack_list = []

  def push(self, val):
     self.__stack_list.append(val)

  def pop(self):
     val = self.__stack_list[-1]
     del self.__stack_list[-1]
     return val

stack_object = Stack()

stack_object.push(3)
stack_object.push(2)
stack_object.push(1)

print(stack_object.pop())
print(stack_object.pop())
print(stack_object.pop())
```

```
class Stack:
    def __init__(self):
        self.__stack_list = []

    def push(self, val):
        self.__stack_list.append(val)

    def pop(self):
        val = self.__stack_list[-1]
        del self.__stack_list[-1]
        return val

stack1 = Stack()
stack2 = Stack()
stack3 = Stack()

stack1.push(1)
stack2.push(stack1.pop() + 1)
stack3.push(stack2.pop() - 2)

print(stack3.pop())
```

Privát – publikus példa:

```
class Auto:
    def __init__(self):
        self.__motor = "V8"          # privát attribútum

    def __motor_check(self):      # privát metódus
        return "Motor OK"

    def inditas(self):
        if self.__motor_check():     # belső használat
            print("Indítás...")

auto = Auto()
# print(auto.__motor)        # AttributeError - nem elérhető!
# auto.__motor_check()    # AttributeError - nem elérhető!
auto.inditas()                    # Ez működik
```

Subclass: issubclass() függvény

```
class Stack:
    pass
class Stack_1(Stack):
    pass

obj = Stack()
obj_1 = Stack_1()

print(issubclass(Stack_1, Stack))          # True
print(Stack_1.__bases__[0].__name__)   # Stack
print(isinstance(obj_1, Stack_1))          # True
print(isinstance(obj_1, Stack))            # True
print(isinstance(obj, Stack_1))            # False
```

Több szülőosztály példa

```
class A:
    pass

class B:
    pass

class C(A, B): # Két szülőosztály adott sorrendben
    pass

print(C.__bases__)       # Tuple
print(C.__bases__[0])  # az 1. szülőosztály
print(C.__bases__[1])  # a 2. szülőosztály
```

```
(<class '__main__.A'>, <class '__main__.B'>)
<class '__main__.A'>
<class '__main__.B'>
```

Subclass példa: AddingStack

```
class Stack:
    def __init__(self):
        self.__stack_list = []

    def push(self, val):
        self.__stack_list.append(val)

    def pop(self):
        val = self.__stack_list[-1]
        del self.__stack_list[-1]
        return val
```

```
class AddingStack(Stack):
    def __init__(self):
        Stack.__init__(self)
        self.__sum = 0

    def get_sum(self):
        return self.__sum

    def push(self, val):
        self.__sum += val
        Stack.push(self, val)

    def pop(self):
        val = Stack.pop(self)
        self.__sum -= val
        return val
```

```
stack_object = AddingStack()

for i in range(5):
    stack_object.push(i)
    print(i, stack_object.get_sum())

for i in range(5):
    print(stack_object.pop(),
          stack_object.get_sum())
```

```
0 0
1 1
2 3
3 6
4 10
```

```
4 6
3 3
2 1
1 0
0 0
```

2. feladat Subclass: CountingStack

Számold meg, hogy hány elemmel történt be-ki művelet a veremben.

```
class Stack:
    def __init__(self):
        self.__stl = []

    def push(self, val):
        self.__stl.append(val)

    def pop(self):
        val = self.__stl[-1]
        del self.__stl[-1]
        return val
```

```
class CountingStack(Stack):
def __init__(self):
    ###         # Definiálj private counter-t

def get_counter(self):
    ###         # Írd ki a számláló aktuális értékét!

def pop(self):
    ###         # A pop metódusban frissítsd a számlálót!

c_stk = CountingStack()
for i in range(100):
    c_stk.push(i)
    c_stk.pop()
print(c_stk.get_counter())
```

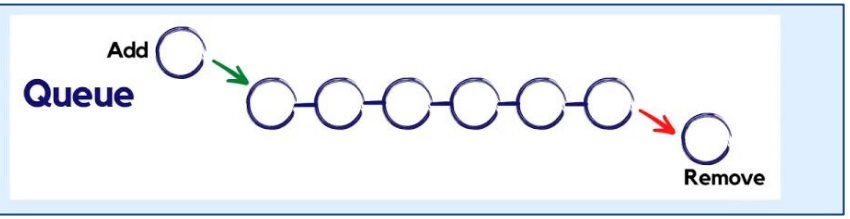
100

3. feladat: A sor OOP implementálása

```
class Queue:
    def __init__(self):
        ###
    def put(self, elem):
        ###
    def get(self):
        ###

que = Queue()
que.put(1)
que.put("dog")
que.put(False)

try:
  for i in range(4):
    print(que.get())
except:
  print("Queue error")
```



https://hackr.io/blog/how-to-use-a-python-deque

```
1
dog
False
Queue error
```

Példa egyedi kivétel osztály készítésére

```
class InvalidAgeError(Exception):
    """Exception, ha a kor nincs az érvényes tartományban"""
    def __init__(self, age, message="Az életkornak a  18 és 40 közötti tartományban kell lenni!"):
        self.age = age
        self.message = message
        super().__init__(self.message)      # Inicializáljuk a szülőosztály konstruktorát a super()-rel az
                                            # Exception.__init__(self, self.message)  helyett  (uaz.)
```

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.set_age(age)
    def set_age(self, age):
        if age < 18 or age > 40:
            raise InvalidAgeError(age)
        self.age = age
    def __str__(self):
        return f"Hallgató neve: {self.name}, Kor: {self.age}"
```

```
try:
    student = Student("Nagy Béla", 50)
except InvalidAgeError as e:
    print(f"Error: {e}")
else: print(student)
```

```
try:
    student = Student("Kiss Anna", 18)
except InvalidAgeError as e:
    print(f"Error: {e}")
else: print(student)
```

Házi feladat: Pontok távolsága Pithagorasz tétellel

A Pont osztály:

- a konstruktor két argumentuma x és y, mindkettő alap-értelmezés szerint 0.0;
- minden attribútumnak privátnak kell lennie;
- az osztály két paraméter nélküli metódust tartalmaz **get_x()** és **get_y()** néven, amelyek a koordinátákat visszaadják;
- az osztály **távolság_from_xy(x,y)** nevű metódusa kiszámítja és visszaadja a távolságot az objektumban tárolt pont, és az (x,y) koordinátájú pont között;
- a **távolság_from_pont(pont)** nevű metódus kiszámítja az objektumban tárolt pont és a paraméterként megadott pont távolságát, ahol a pont paraméter a Pont osztály egy másik objektumaként van megadva.

Írd meg a hiányzó kódokat, futtasd a programot, ellenőrizd, hogy a kimenet a megadott-e. Teszteld egyéb esetekre is.

```
import math
class Pont:
   def __init__(self, x=0.0, y=0.0):
      ###

   def get_x(self):
      ###

   def get_y(self):
      ###

   def távolság_from_xy(self, x, y):
      ###

   def távolság_from_pont(self, pont):
      ###
pont1 = Pont(0, 0)
pont2 = Pont(1, 1)
print(pont1.távolság_from pont(pont2))
print(pont2.távolság_from_xy(2, 0))
```

```
1.4142135623730951
1.4142135623730951
```

Köszönöm a figyelmet!

Konzultáció

Minden csütörtökön 18:00 – 20:00